
**TRIBHUVAN UNIVERSITY**

**INSTITUTE OF ENGINEERING**

**THAPATHALI CAMPUS**

**Minor Mid-Defense Report**

**ON**

# GEO-LOCATING USERS IN MOUNTAINS BY DETECTING SKYLINES FROM CAPTURED PHOTOS

**Submitted By:**

| | |
|---|---|
| Bibha Ray | (THA080BCT009) |
| Prasiddha Raj Poudel | (THA080BCT027) |
| Reema Kasula | (THA080BCT032) |

**Submitted To:**

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In partial fulfillment for the award of the

Bachelors Degree in Electronics, Communication and Information Engineering

**Under the Supervision of**

Er. Shanta Maharjan

July 2026


**TRIBHUVAN UNIVERSITY**

**INSTITUTE OF ENGINEERING**

**THAPATHALI CAMPUS**

**Minor Mid-Defense Report**

**ON**

# GEO-LOCATING USERS IN MOUNTAINS BY DETECTING SKYLINES FROM CAPTURED PHOTOS

**Submitted By:**

| | |
|---|---|
| Bibha Ray | (THA080BCT009) |
| Prasiddha Raj Poudel | (THA080BCT027) |
| Reema Kasula | (THA080BCT032) |

**Submitted To:**

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

July 2026

**ACKNOWLEDGEMENT**

We would like to thank Institute of Engineering, Tribhuvan University, for incorporating minor project within the curriculum of Bachelor's in Electronics, Communication, and Information Engineering.

Special thanks to our supervisor, **Er. Shanta Maharjan**, for guidance, valuable feedback, and constant support throughout the project.

We would like to express our appreciation to the Department of Electronics and Computer Engineering, Thapathali Campus, for their continuous assistance, and recommendations during project selection. This has significantly helped in enabling our professional development.

We extend our sincere thanks to all the teachers, friends, and both direct and indirect contributors for their valuable insights, recommendations, and encouragement throughout this journey.

Bibha Ray (THA080BCT009)
Prasiddha Raj Poudel (THA080BCT027)
Reema Kasula (THA080BCT032)

## ABSTRACT

Finding a person's location in remote mountain regions can be difficult when satellite navigation is unavailable or unreliable. In deep valleys, surrounding mountains can block Global Navigation Satellite System (GNSS) signals, while mobile network coverage is often limited. This creates a need for an alternative method that can estimate location without depending on satellite positioning or internet connectivity. This project presents an offline visual geolocation system for the Khumbu region of Nepal that estimates the location of a photograph using the surrounding mountain skyline. A Digital Elevation Model (DEM) is used to generate a database of reference skylines from regularly spaced viewpoints across the study area. When a user captures a photograph, the skyline is extracted by separating the sky from the terrain using a lightweight semantic segmentation model based on the U-Net architecture with a MobileNetV3 encoder. Because labelled Himalayan images are scarce, the segmentation model is trained on GeoPose3K's predefined training split (3,000 real images) combined with 300 synthetically generated images created from the terrain model under different illumination and sky appearance conditions. The extracted skyline is then compared with the reference database using a two-stage hierarchical matching process. A fast coarse search identifies the most likely candidates, followed by a finer alignment stage that improves robustness against small viewpoint and camera differences. The proposed method was evaluated using 300 synthetic test images covering different locations and environmental conditions within the study area. Under grid-aligned synthetic evaluation with optional height-filtered candidate pruning, where query viewpoints coincide with reference grid locations and can be filtered using approximate height information, the system correctly localized 94.67% of the test images within 1 km of the true location and 84.00% within 100 m. These results suggest that mountain skylines provide reliable geographic information for visual localization in the Khumbu region. Although the current evaluation is based on synthetic imagery, the framework establishes a practical foundation for lightweight offline mountain navigation and can be extended through future testing with real-world photographs.

***Keywords***— visual geolocation, mountain skyline, DEM, semantic segmentation, synthetic data, Himalayan navigation, offline localization

**TABLE OF CONTENTS**

**ACKNOWLEDGEMENT** . . . . . **i**

**ABSTRACT** . . . . . **ii**

**LIST OF FIGURES** . . . . . **v**

**LIST OF TABLES** . . . . . **vi**

**LIST OF ALGORITHMS** . . . . . **vii**

**LIST OF ABBREVIATIONS** . . . . . **viii**

**1. Introduction** . . . . . **1**
- 1.1. Background . . . . . 1
- 1.2. Problem Statement . . . . . 2
- 1.3. Motivation . . . . . 2
- 1.4. Objectives . . . . . 3
- 1.5. Scope and Limitations . . . . . 3
- 1.6. Applications . . . . . 3

**2. Literature Review** . . . . . **4**
- 2.1. Visual Geolocation in Mountainous Terrain . . . . . 4
- 2.2. DEMs . . . . . 5
- 2.3. Improving Skyline Matching . . . . . 5
- 2.4. Skyline Extraction . . . . . 7
- 2.5. Synthetic Data for Mountain Geolocation . . . . . 8
- 2.6. Comparison of Existing Methods . . . . . 9
- 2.7. Research Gap . . . . . 10

**3. Requirements Analysis** . . . . . **12**
- 3.1. Functional Requirements . . . . . 12
- 3.2. Non-Functional Requirements . . . . . 12
- 3.3. Development Environment . . . . . 12

**4. System Architecture and Methodology** . . . . . **14**
- 4.1. System Overview . . . . . 14
- 4.2. Study Area . . . . . 15
- 4.3. Reference Skyline Database . . . . . 16
- 4.4. Synthetic Query Dataset . . . . . 19
- 4.5. Skyline Segmentation . . . . . 20

4.6. Skyline Extraction . . . . . 22
4.7. Skyline Matching . . . . . 24

**5. Implementation Details . . . . . 27**
5.1. Reference Skyline Database Implementation . . . . . 27
5.2. Synthetic Dataset Generation . . . . . 28
5.3. Sky Segmentation Implementation . . . . . 29
5.4. Skyline Extraction Implementation . . . . . 30
5.5. Skyline Matching Implementation . . . . . 30
5.6. Evaluation Implementation . . . . . 32

**6. Results and Analysis . . . . . 34**
6.1. Evaluation Setup . . . . . 34
6.2. Synthetic Query Images . . . . . 35
6.3. Segmentation Training Result . . . . . 36
6.4. Localization Accuracy . . . . . 37
6.5. Qualitative Examples . . . . . 39
6.6. Summary . . . . . 41

**7. Remaining Tasks . . . . . 42**
7.1. Google Street View Validation . . . . . 42
7.2. Automatic Failure Detection . . . . . 42
7.3. Landmark Association . . . . . 42
7.4. Off-Grid Robustness . . . . . 43
7.5. Summary . . . . . 43

**APPENDICES . . . . . 44**

**A. PROJECT TIMELINE . . . . . 44**
A.1. Gantt Chart . . . . . 44

**REFERENCES . . . . . 45**

## LIST OF FIGURES

Figure 4-1. Overall workflow of the proposed skyline-based localization system. 15
Figure 4-2. Study area used for reference skyline generation and system evaluation. 16
Figure 4-3. Reference skyline database generation process. . . . . 17
Figure 4-4. Synthetic dataset generation process. . . . . 19
Figure 4-5. Skyline segmentation process. . . . . 21
Figure 4-6. Skyline extraction process. . . . . 23
Figure 4-7. Skyline matching process. . . . . 25
Figure 6-1. Reference skyline database preview used during matching. . . . . 35
Figure 6-2. Examples of synthetic query images used for evaluation. . . . . 36
Figure 6-3. Training and validation curves of the segmentation model. . . . . 37
Figure 6-4. Localization accuracy at different distance thresholds. . . . . 38
Figure 6-5. Representative localization visualizations from the grid-aligned test set. 40
Figure A-1. Project Gantt Chart . . . . . 44

## LIST OF TABLES

Table 2-1. Comparison of representative mountain geolocation methods. . . . . 10
Table 3-1. Functional requirements . . . . . 12
Table 3-2. Non-functional requirements . . . . . 12
Table 3-3. Development environment . . . . . 13
Table 6-1. Evaluation setup used in this chapter. . . . . 34
Table 6-2. Localization accuracy of the proposed system. . . . . 37

## LIST OF ALGORITHMS

1. Reference Skyline Generation . . . . . 18
2. Skyline Extraction from Binary Sky Mask . . . . . 31
3. Two-Stage Skyline Matching . . . . . 33

## LIST OF ABBREVIATIONS

| | |
|---|---|
| DEM | Digital Elevation Model |
| DTW | Dynamic Time Warping |
| FFT | Fast Fourier Transform |
| FOV | Field of View |
| GNSS | Global Navigation Satellite System |
| GPS | Global Positioning System |
| GSV | Google Street View |
| IoU | Intersection over Union |

# 1. INTRODUCTION

## 1.1. Background

Knowing one's location is essential for safe travel in mountainous regions. Modern navigation mainly depends on Global Navigation Satellite Systems (GNSSs), such as Global Positioning System (GPS), together with online maps. These technologies work well in many places, but their performance becomes less reliable in deep mountain valleys. High mountain ridges can block satellite signals, and mobile network coverage is often unavailable in remote areas. As a result, trekkers, climbers, researchers, and rescue teams may not always have reliable location information when it is needed most.

Unlike cities, mountains contain very few man-made landmarks that can be recognized automatically. However, mountains have one natural feature that changes very little over time: the skyline. The outline formed where the mountain meets the sky remains almost the same even when seasons, weather, vegetation, or lighting conditions change. Although, its visibility might be affected due to them. Because of this stability, the skyline can act as a natural landmark for estimating location.

Recent research has shown that skylines extracted from photographs can be compared with skylines generated from Digital Elevation Models (DEMs). Since DEMs describe the shape and elevation of the terrain, they can be used to generate reference skylines from many different viewpoints without visiting each location. A photograph can then be matched against these references to estimate where it was taken.

Unlike satellite-based positioning, this approach depends only on the information already present in the photograph and the terrain model. As long as the mountain skyline is visible, the photograph itself becomes a source of geographic information. This makes skyline-based localization particularly attractive for remote mountain regions where navigation and communication infrastructure may be limited.

Most previous studies have focused on mountain regions such as the European Alps or nearby hilly terrain in China. These environments differ from the Himalayas in several ways. The Khumbu region contains much larger elevation differences, fewer labelled photographs, and long mountain views that behave differently from nearby hills. Methods developed for other regions therefore may not generalize directly without modification.

This project investigates skyline-based visual geolocation for the Khumbu region of Nepal. By combining terrain data with computer vision, the proposed system estimates the location of a photograph using only the surrounding mountain skyline. The system is designed to operate without relying on continuous internet connectivity and aims

to provide an additional source of location information when conventional navigation methods become unreliable.

### 1.2. Problem Statement

Reliable positioning remains a challenge in remote mountain environments. Although GNSS receivers are widely available, their accuracy may decrease when surrounding terrain reduces satellite visibility and blocks satellite signals. Internet-based navigation services are also difficult to use where mobile network coverage is limited.

Visual geolocation provides an alternative by estimating location directly from a photograph. Existing skyline-based methods have shown promising results, but most were developed for environments that differ from the Himalayas. Many assume the availability of large image datasets, nearby mountain ridges, or computational resources that are not suitable for lightweight offline systems.

The Khumbu region introduces additional challenges. Labelled mountain images are scarce, mountain peaks are often several kilometres away from the camera, and weather conditions frequently reduce skyline visibility through clouds, haze, and changing illumination. These factors make accurate skyline extraction and matching more difficult while also reducing the effectiveness of methods designed for other environments.

Therefore, there is a need for a visual geolocation system designed specifically for the Himalayan environment that can estimate location from a mountain photograph without relying on continuous GNSS availability or internet connectivity.

### 1.3. Motivation

The Khumbu region is one of the world's most popular trekking and mountaineering destinations. Every year, thousands of visitors travel through its valleys, where reliable location information is important for navigation and safety. An alternative positioning method could also assist environmental surveys, mountain research, and the analysis of landscape photographs captured in remote areas.

At the same time, freely available terrain data and recent advances in computer vision have made skyline-based localization increasingly practical. DEMs allow realistic reference skylines to be generated without visiting every viewpoint, while modern image segmentation techniques can accurately separate the sky from the surrounding mountains.

These developments create an opportunity to build a lightweight visual geolocation system for the Himalayas. Rather than replacing conventional navigation methods, the proposed system is intended to provide an additional source of location information when satellite navigation or communication services become unreliable.

## 1.4. Objectives

The main objective of this project is to develop and evaluate an offline visual geolocation system for the Khumbu region of Nepal using mountain skylines.

The specific objectives are:

1. To generate a reference database of mountain skylines from a DEM.
2. To develop a lightweight skyline extraction and matching pipeline capable of estimating the location of a photograph.

## 1.5. Scope and Limitations

### 1.5.1. Scope

This project focuses on skyline-based visual geolocation within the Khumbu region of Nepal. The reference database is generated from the Copernicus GLO-30 DEM using viewpoints sampled on a regular grid. The system accepts a single landscape photograph as input and estimates the camera location by matching its skyline with the reference database.

### 1.5.2. Limitations

The proposed system assumes that the mountain skyline is clearly visible in the input image. Heavy cloud cover, dense foreground vegetation, nearby buildings, and severe occlusions may reduce localization accuracy. The segmentation model is trained mainly using synthetic images because large labelled Himalayan datasets are not available. The system is evaluated primarily using synthetic test images, while extensive validation using real-world photographs remains future work.

## 1.6. Applications

The proposed framework has the following applications:

- Estimating the approximate location of a mountain photograph using the surrounding skyline.
- Assisting mountain photography by estimating where a landscape photograph was taken.
- Supporting environmental and geographic field studies in remote mountain regions.
- Serving as a foundation for future offline mountain localization systems.

## 2. LITERATURE REVIEW

### 2.1. Visual Geolocation in Mountainous Terrain

Visual geolocation estimates the location where a photograph was taken using information contained in the image. In urban environments, buildings, roads, and other man-made structures provide many distinctive landmarks that can be matched with reference images. Mountain environments are much more challenging because natural landscapes contain fewer unique visual features. Their appearance also changes with weather, lighting, snow cover, and seasonal vegetation, making conventional image matching less reliable.

Researchers found that one feature remains relatively stable despite these changes: the mountain skyline. Although snow and vegetation may change throughout the year, the boundary between the mountains and the sky is determined by the terrain itself. This makes the skyline a reliable geographic feature for estimating camera position. Early studies demonstrated that mountain skylines generated from DEMs could be aligned with photographs to recover the camera location and orientation [1, 2]. These works established the foundation for later skyline-based localization methods.

A major step towards large-scale visual geolocation was presented by [3]. Instead of matching individual mountain peaks, they represented panoramic skylines using contour descriptors extracted from DEM-generated terrain models. Since a photograph usually contains only part of the surrounding horizon, the system searched for reference panoramas that contained the observed skyline rather than requiring a complete match. Their experiments over approximately 40,000 $km^2$ of Switzerland demonstrated that skyline shape alone contains sufficient information for large-scale localization.

Building on this work, [4] observed that similar skyline shapes could sometimes appear at different locations, especially when only a small portion of the skyline was visible. To reduce false matches, they improved both skyline extraction and candidate verification. After retrieving the most likely candidate locations, they compared the complete skyline with the rendered terrain model before selecting the final match. This additional verification step significantly improved localization accuracy while maintaining efficient retrieval.

These studies established skyline matching as a practical approach for mountain geolocation. They also demonstrated the value of DEMs, which make it possible to generate reference skylines without requiring photographs from every possible location. As the field matured, researchers shifted their attention from demonstrating

the feasibility of skyline matching to improving its robustness under more challenging imaging conditions and different types of terrain.

### 2.2. DEMs

A DEM is a digital representation of the Earth's terrain. In skyline-based geolocation, a DEM is used to generate synthetic views of the landscape from different virtual camera positions. These rendered views provide the reference skylines that are later compared with photographs. As a result, the quality of the DEM directly affects the accuracy of the generated skyline.

Several global DEMs are available for mountainous terrain, including NASADEM, ALOS AW3D30, and Copernicus GLO-30. Each has strengths depending on the application. NASADEM generally provides better ground elevation in densely forested regions because its radar measurements penetrate vegetation more effectively. AW3D30 performs well in some rugged mountain landscapes but may introduce artificial terrain artefacts. Copernicus GLO-30 provides a more detailed representation of ridges and valleys, making it well suited for applications that depend on terrain morphology rather than vegetation structure [5].

The Khumbu region is dominated by steep mountain terrain where the overall shape of ridges and skylines is more important than modelling the ground beneath dense vegetation. Since this research generates synthetic skylines from terrain data, preserving ridge geometry is the primary requirement. Previous evaluations have shown that Copernicus GLO-30 provides detailed and consistent representations of mountainous terrain, making it suitable for skyline generation and visual geolocation [5].

For these reasons, Copernicus GLO-30 was selected to generate the synthetic mountain views and reference skylines used throughout this research.

### 2.3. Improving Skyline Matching

Although skyline matching proved to be effective, researchers soon identified several practical limitations. In real-world photographs, the skyline is not always clearly visible. Clouds, haze, vegetation, and foreground objects may partially hide the mountain boundary. In addition, different locations can sometimes produce similar skyline shapes, making it difficult to distinguish between nearby viewpoints. These challenges motivated the development of more robust skyline representations and matching techniques.

One important limitation was identified by [6]. Earlier methods relied primarily on the outer skyline, assuming that it could always be extracted reliably from the input image. However, the authors showed that weather conditions, image quality, and atmospheric

effects often make the skyline incomplete or unreliable. To overcome this problem, they incorporated mountain ridge lines in addition to the skyline. Ridge lines were extracted from depth discontinuities in DEM-generated terrain models and provided additional geometric information when the skyline alone was insufficient. The method also estimated the camera roll angle during localization, showing that even small rotations could noticeably reduce matching accuracy if ignored. By combining skyline and ridge information, the system produced more reliable localization under challenging viewing conditions.

While [6] improved the robustness of skyline matching, their method was developed primarily for large mountain ranges where the skyline changes gradually with camera movement. A different challenge arises in hilly landscapes, where the skyline is much closer to the observer. In these environments, even small changes in camera position can significantly alter the shape of the skyline.

To address this problem, [7] proposed a skyline representation designed specifically for nearby hills. Instead of using only the outer mountain boundary, they introduced *lapel points*, which are formed where internal ridge lines intersect the skyline. These points act as stable landmarks that make neighbouring skylines easier to distinguish. The authors also compared skylines at multiple image scales, allowing the system to compensate for small differences in viewing distance that would otherwise lead to incorrect matches. Their results demonstrated that incorporating additional terrain structure significantly improved localization accuracy in hilly environments.

As skyline databases became larger, the computational cost of searching every reference skyline also increased. Rather than improving the skyline representation itself, [8] focused on making the retrieval process more efficient. Similar skylines were first grouped into clusters, reducing the number of candidate matches that needed to be examined. The authors also showed that the geometric relationship between matched skyline features could be used to estimate the camera orientation directly from the image, reducing the dependence on external orientation sensors. These improvements reduced localization time while maintaining comparable accuracy.

Although these methods advanced skyline matching considerably, they were developed for landscapes that differ from the Himalayan environment. The European studies focused on the Alps, while the work of [8] considered densely hilly terrain where nearby viewpoints produce rapidly changing skylines. In the Himalayas, mountain peaks are typically much farther from the observer, causing the skyline to remain relatively stable over neighbouring viewpoints. Consequently, techniques designed specifically for nearby hills, such as dense lapel-point representations and fine-scale viewpoint sampling,

become less important. Instead, robust skyline extraction and efficient matching are more critical for reliable localization in high mountain regions.

A notable benchmark for visual geolocation in mountainous terrain is the GeoPose3K dataset [9], which contains mountain images with associated ground truth locations. This dataset has been used by multiple research groups to evaluate and compare localization methods under real-world conditions, making it useful for situating new approaches within the broader field.

### 2.4. Skyline Extraction

The success of any skyline-based localization system depends on how accurately the skyline can be extracted from an input image. Even if a reference database and matching algorithm are highly accurate, errors introduced during skyline extraction can propagate through the rest of the localization pipeline. In practice, extracting the skyline is often challenging because outdoor images contain clouds, haze, trees, buildings, and varying illumination that obscure the true boundary between the mountains and the sky.

One of the earliest studies to address this problem was presented by [10]. The authors observed that conventional edge detectors faced a trade-off between detection and localization. A detector tuned to identify only strong edges often missed parts of the skyline, while one tuned to capture fine details also detected many unwanted edges produced by clouds, shadows, and vegetation. To overcome this limitation, they generated edge maps at two different scales. A coarse-scale edge map was used to identify reliable skyline points, while a finer-scale edge map traced the complete skyline between these points. This two-stage approach produced more accurate skyline extraction under difficult outdoor conditions.

Although edge-based methods improved skyline detection, they relied heavily on carefully selected image-processing parameters. Their performance often decreased when weather conditions or lighting changed significantly. Rather than manually designing filters to detect the skyline, later research explored learning-based approaches that could adapt automatically to different image characteristics.

A resource-efficient learning-based method was proposed by [11]. Instead of using a deep neural network, the authors trained a bank of lightweight linear filters to distinguish skyline edges from other image edges. During inference, the local image structure determined which filter should be applied at each pixel, allowing the method to suppress unwanted edges while preserving the mountain boundary. A dynamic programming algorithm was then used to recover a continuous skyline from the filtered responses. This approach achieved accuracy comparable to more complex learning-based

methods while requiring considerably less computational power, making it suitable for resource-constrained platforms such as mobile devices, unmanned air vehicles, and planetary rovers.

Despite these improvements, both edge-based and shallow learning methods still treated skyline extraction primarily as an edge detection problem. Recent advances in computer vision instead formulate skyline extraction as a semantic segmentation task. Rather than identifying individual edges, semantic segmentation classifies every pixel in the image as either sky or terrain. The skyline is then obtained directly from the boundary between the two regions. Since the prediction is based on information from the entire image instead of local gradients alone, semantic segmentation is generally more robust to clouds, shadows, vegetation, and varying illumination.

This shift towards semantic segmentation has enabled more reliable skyline extraction in challenging mountain environments. Encoder-decoder architectures such as U-Net combine high-level semantic information with fine image details, allowing accurate pixel-level segmentation while preserving object boundaries. These properties make semantic segmentation particularly suitable for mountain geolocation, where precise skyline boundaries are essential for reliable matching with DEM-generated reference skylines.

### 2.5. Synthetic Data for Mountain Geolocation

As visual geolocation methods became increasingly dependent on machine learning, the availability of labelled training data emerged as a major challenge. Unlike urban environments, remote mountain regions contain relatively few publicly available photographs, and manually annotating mountain skylines is both time-consuming and labour-intensive. This lack of training data limits the performance of learning-based skyline extraction and localization methods.

To address this problem, [12] proposed CrossLocate, a cross-modal visual geolocation framework that combines real photographs with synthetic terrain rendered from DEMs. Instead of comparing only real images, the authors rendered depth maps and terrain views from many virtual camera positions. A deep neural network was then trained to learn a shared feature representation between rendered terrain and real photographs. This approach allowed the system to localize images even in regions where very few real training photographs were available.

An important finding of their work was that synthetic data is not simply a replacement for real images. The authors showed that there is a noticeable gap between synthetic and real image domains, meaning that models trained only on one type of data do

not generalize well to the other. By jointly learning from both domains, CrossLocate achieved significantly better localization performance over large mountainous regions than methods relying on a single data source.

More recently, [13] extended this idea through a complete cross-modal localization framework designed for natural environments without GNSS information. Their system combined semantic segmentation, DEM-generated reference data, and efficient database retrieval to localize images directly from terrain features. Rather than relying solely on skyline contours, the framework learned richer semantic representations while maintaining practical retrieval speeds for large reference databases. Their results demonstrated that combining semantic understanding with terrain information can improve both localization accuracy and retrieval efficiency.

These studies demonstrate that synthetic terrain generated from DEMs provides an effective solution to the limited availability of labelled mountain imagery. Synthetic data makes it possible to generate training samples and reference views for locations where few or no photographs exist. This is particularly valuable in remote mountain regions such as the Himalayas, where collecting large, well-annotated image datasets is difficult. Consequently, synthetic data has become an important component of modern mountain geolocation systems.

### 2.6. Comparison of Existing Methods

Table 2-1 summarizes the main approaches discussed in this chapter. Early studies focused on demonstrating that mountain skylines could be used for localization. Later research improved skyline matching, developed more robust skyline extraction methods, and introduced synthetic terrain to overcome the limited availability of mountain image datasets. Together, these studies established skyline-based localization as a practical approach for natural environments.

Table 2-1. Comparison of representative mountain geolocation methods.

| **Study** | **Main Contribution** | **Terrain** | **Limitation** |
|---|---|---|---|
| Baatz et al. [3] | Contour-based skyline matching | Alps | Requires reliable skyline extraction |
| Saurer et al. [4] | Improved skyline verification | Alps | Designed mainly for Alpine terrain |
| Chen et al. [6] | Skyline and ridge matching | Mountain regions | Higher computational complexity |
| Pan et al. [7] | Lapel points for nearby hills | Hilly terrain | Assumes rapidly changing skylines |
| Pan et al. [8] | Efficient skyline retrieval | Hilly terrain | Sensitive to pseudo skylines |
| Yang et al. [10] | Robust edge-based skyline extraction | Mountain regions | Sensitive to image conditions |
| Ahmad et al. [11] | Lightweight skyline extraction | Mountain regions | Still relies on edge responses |
| Tomesek et al. [12] | Cross-modal learning using synthetic terrain | Large-scale mountains | Requires deep learning and large datasets |
| Liu et al. [13] | Cross-modal semantic localization | Natural environments | Computationally intensive |

## 2.7. Research Gap

Previous research has shown that mountain skylines provide reliable information for visual geolocation. Researchers have improved skyline matching, developed more robust skyline extraction methods, and demonstrated the usefulness of synthetic terrain generated from DEMs. Together, these advances have established skyline-based localization as a practical alternative when conventional image matching is unreliable.

Despite this progress, most existing systems have been developed for environments that differ from the Himalayas. Many studies focus on the European Alps, where large image datasets are available and mountain viewpoints are relatively well documented. Other work targets nearby hilly terrain, where small changes in camera position produce significant changes in skyline shape. These assumptions do not necessarily hold in the Khumbu region, where distant mountain ranges create more stable skyline profiles over neighbouring viewpoints.

Another limitation is that many recent approaches rely on computationally expensive deep learning models or large image databases. While these methods achieve high localization accuracy, they are less suitable for lightweight deployment on mobile

devices operating in remote mountain environments. In addition, satellite navigation may be unreliable in deep valleys where surrounding terrain blocks GNSS signals, making image-based localization an attractive alternative.

These observations highlight the need for a localization system designed specifically for the Himalayan environment. Such a system should operate without internet connectivity, generate reference data directly from terrain models, and perform efficient skyline matching while remaining practical for deployment on resource-constrained devices. This research addresses these requirements by combining semantic skyline extraction, synthetic terrain generation, and hierarchical skyline matching for visual geolocation in the Khumbu region.

# 3. REQUIREMENTS ANALYSIS

This chapter defines the main requirements of the proposed visual geolocation system. It describes the functions the system is expected to perform, the quality requirements that guided its design, and the software used during development.

## 3.1. Functional Requirements

Table 3-1 summarizes the main functional requirements of the proposed system.

Table 3-1. Functional requirements

| ID | Requirement |
|---|---|
| **FR-01** | The system shall generate reference skyline profiles from a DEM. |
| **FR-02** | The system shall accept a single landscape photograph as input. |
| **FR-03** | The system shall extract the mountain skyline from the input image. |
| **FR-04** | The system shall compare the extracted skyline with the reference database. |
| **FR-05** | The system shall estimate the camera location from the best matching skyline. |

## 3.2. Non-Functional Requirements

Table 3-2 summarizes the non-functional requirements considered during the design of the system.

Table 3-2. Non-functional requirements

| ID | Requirement |
|---|---|
| **NFR-01** | The system shall perform localization without requiring an internet connection. |
| **NFR-02** | The system shall use a lightweight segmentation model to reduce computational cost. |
| **NFR-03** | The system shall produce consistent results for the same input image. |
| **NFR-04** | The system shall allow the reference skyline database to be regenerated for a different study area. |
| **NFR-05** | The system shall use a modular pipeline so that individual components can be updated independently. |

## 3.3. Development Environment

The proposed system was developed using the software listed in Table 3-3.

Table 3-3. Development environment

| Software | Purpose |
|---|---|
| Python | Main programming language |
| PyTorch | Training the segmentation model |
| OpenCV | Image processing |
| NumPy | Numerical computation |
| SciPy | FFT correlation for skyline matching |
| Rasterio | Reading and processing DEM data |
| HORAYZON | Generating reference skylines |
| Google Colab | Training the segmentation model |
| LaTeX | Project report preparation |

# 4. SYSTEM ARCHITECTURE AND METHODOLOGY

## 4.1. System Overview

The proposed system estimates the location of a mountain image by comparing its skyline with a database of reference skylines generated from a DEM. Before localization, the system prepares the reference skyline database and trains the skyline segmentation model using a synthetic dataset. During localization, the input image is processed to extract its skyline, which is then matched against the reference database to estimate the camera location.

The system consists of five main modules. The first module generates the reference skyline database from the terrain model. The second module generates a synthetic dataset used to train the segmentation model. The third module separates the sky from the terrain in the query image. The fourth module converts the segmented mask into a skyline profile. The final module compares the extracted skyline with the reference database and returns the best matching location.

Figure 4-1 shows the complete workflow of the proposed system.

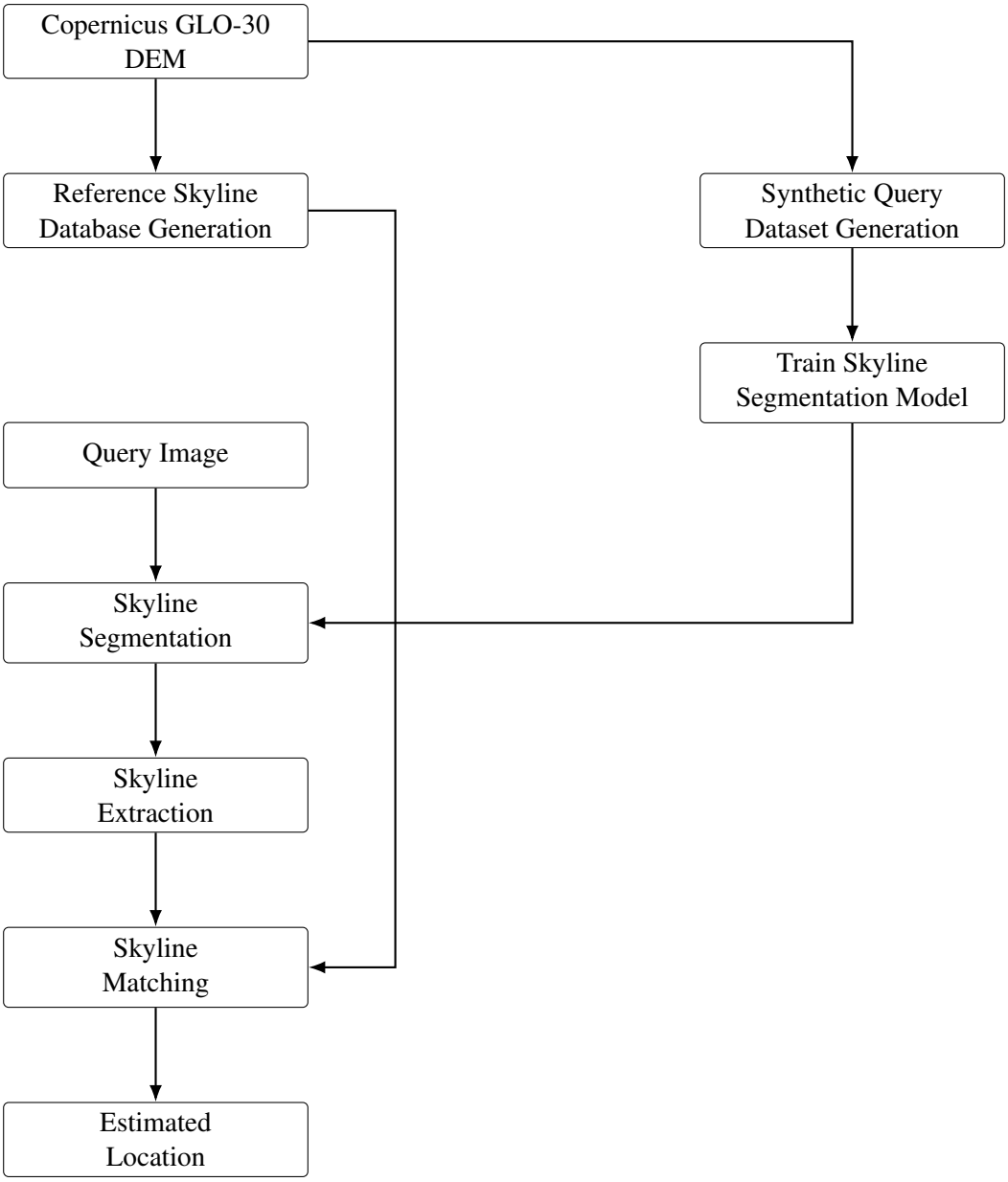



Figure 4-1. Overall workflow of the proposed skyline-based localization system.

The preparation modules are executed once for the selected study area. The generated skyline database and trained segmentation model are then reused for every localization query. During localization, only the query image is required as input. The estimated location is obtained by matching the extracted skyline against the reference skyline database.

### 4.2. Study Area

The proposed system is developed for the Khumbu region of eastern Nepal. This region contains steep terrain, deep valleys, and distinctive mountain skylines that are well suited

for skyline-based localization. The study area also includes several popular trekking routes where GNSS signals may be unreliable because of terrain obstruction.

Terrain information is obtained from the Copernicus GLO-30 DEM. The dataset provides an approximate spatial resolution of 30 m and is used throughout the project. The same terrain model is used to generate the reference skyline database and the synthetic training dataset. Using a single terrain source keeps every stage of the system consistent.

The study area is defined before database generation. Only viewpoints located inside the selected boundary are used when generating reference skylines. Restricting the search area reduces the database size while ensuring that every reference skyline corresponds to a valid location within the study region.

The blue line shown in Figure 4-2 represents the available Google Street View coverage used for future real-world validation.

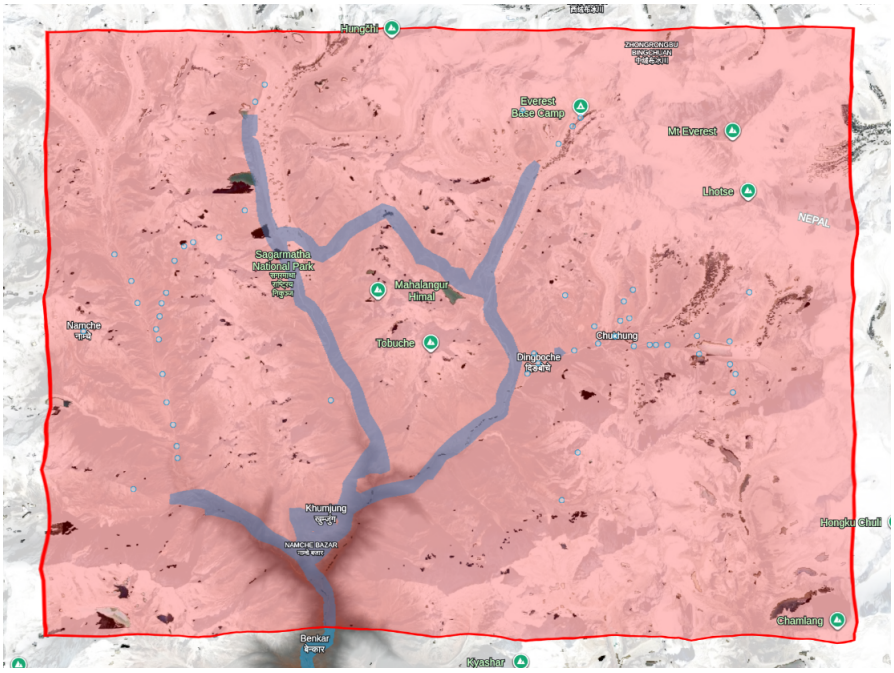



Figure 4-2. Study area used for reference skyline generation and system evaluation.

### 4.3. Reference Skyline Database

The reference skyline database contains the skyline profile associated with every valid viewpoint in the study area. During localization, the extracted skyline from the query image is compared against this database to estimate the camera location. Since the database is generated only once, the computational cost of skyline generation does not affect localization time.

The database generation process begins by selecting viewpoints across the study area. A skyline profile is generated for every viewpoint using the terrain model. Viewpoints that do not contain sufficient terrain variation are removed before the remaining profiles are stored in the database.

Figure 4-3 illustrates the complete database generation process.

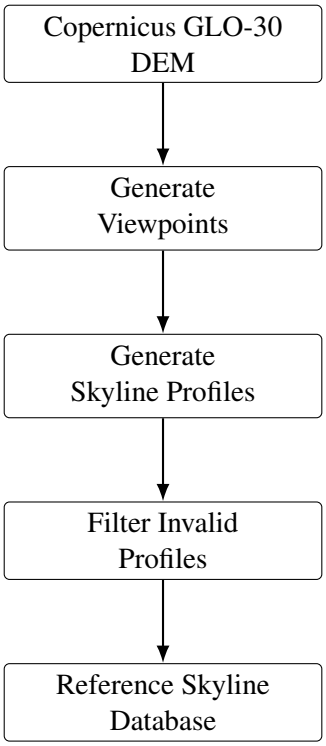



Figure 4-3. Reference skyline database generation process.

### 4.3.1. Viewpoint Generation

The study area is sampled using a regular grid. Each grid point represents a candidate camera location from which a skyline profile is generated. The spacing between neighbouring viewpoints is fixed during database generation so that the study area is sampled uniformly.

The elevation of every viewpoint is obtained directly from the DEM. The viewpoint coordinates are also recorded because they are required after skyline matching to recover the estimated camera location.

### 4.3.2. Skyline Generation

A skyline profile is generated for every viewpoint using the surrounding terrain. The skyline represents the highest visible terrain elevation in every viewing direction.

The horizon is sampled over the complete 360° field of view. For every azimuth direction, the terrain elevation angle is computed from the observer to the visible terrain. The highest elevation angle becomes the skyline value for that direction.

The elevation angle is computed as

$$\theta = \tan^{-1}\left(\frac{h_t - h_o}{d}\right), \tag{4-1}$$

where $h_t$ is the terrain elevation, $h_o$ is the observer elevation, and $d$ is the horizontal distance between the observer and the terrain point. Equation (4-1) is written in its flat-Earth form for clarity. In the implemented pipeline, Earth curvature and atmospheric refraction—which can introduce $\sim 0.1°$ of elevation angle error at 30 km—are corrected internally by HORAYZON during skyline generation.

Repeating this process for all azimuth directions produces a one-dimensional skyline profile that describes the visible terrain surrounding the viewpoint.

Algorithm 1 summarizes the skyline generation procedure.

**Algorithm 1** Reference Skyline Generation

**Require:** Observer location, terrain model
**Ensure:** Skyline profile
1: **for all** azimuth directions **do**
2: Trace the terrain profile
3: Compute the elevation angle of visible terrain points
4: Select the maximum elevation angle
5: Append the value to the skyline profile
6: **end for**
7: **return** the skyline profile

### 4.3.3. Skyline Filtering

Not every generated skyline is suitable for localization. Viewpoints located on flat terrain often produce skyline profiles with little variation. These profiles appear similar at many different locations and reduce matching accuracy.

Each skyline profile is evaluated before being added to the database. Profile quality is assessed using simple geometric characteristics, such as skyline variability and peak prominence. Nearly flat skylines with little discriminative information are removed before storage.

Removing unsuitable viewpoints reduces the database size while improving the distinctiveness of the remaining skyline profiles.

### 4.3.4. Database Storage

The remaining skyline profiles are stored together with the corresponding viewpoint information. Each database entry contains the viewpoint coordinates and the associated skyline profile.

The database is loaded during localization and searched using the extracted skyline profile from the query image. Since every stored profile corresponds to a known viewpoint, the location of the best matching profile becomes the estimated camera location.

## 4.4. Synthetic Query Dataset

A labelled dataset is required to train the skyline segmentation model. Since no suitable labelled mountain image dataset is available for the study area, the training data is generated automatically. The synthetic dataset provides paired RGB images and ground truth sky masks without manual annotation.

The dataset is generated from the same terrain model used for the reference skyline database. This ensures that the terrain observed during training is consistent with the terrain used during localization. Every generated image has a corresponding binary sky mask and ground truth viewpoint information.

Figure 4-4 shows the dataset generation process.

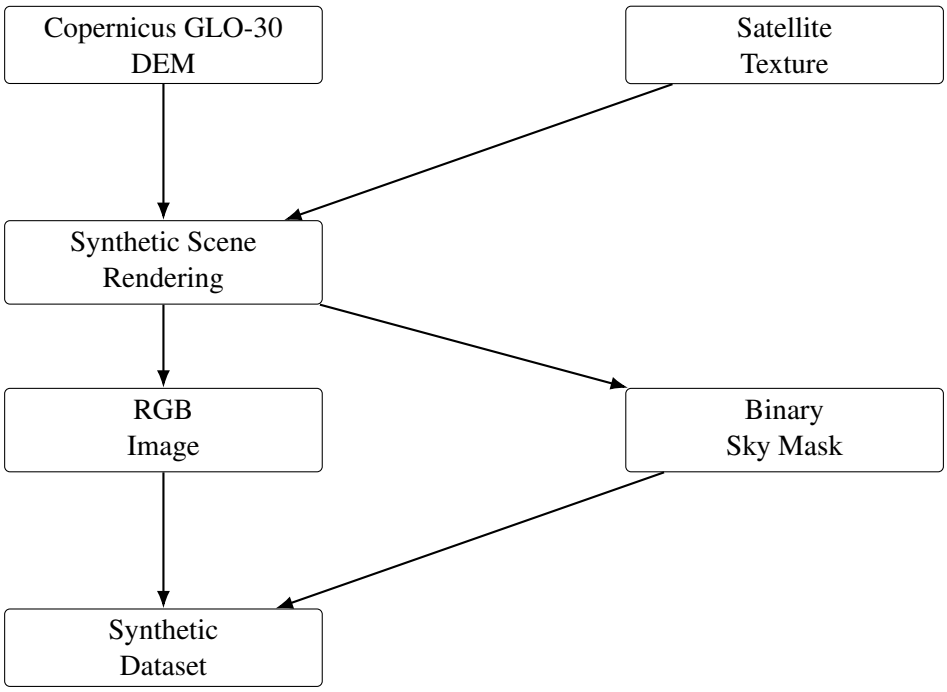



Figure 4-4. Synthetic dataset generation process.

### 4.4.1. Scene Generation

Synthetic mountain scenes are rendered using the terrain model and satellite imagery. The terrain geometry is obtained from the Copernicus GLO-30 DEM, while the satellite image is mapped onto the terrain surface as a texture.

The virtual camera is positioned at viewpoints distributed throughout the study area. For each viewpoint, the terrain is rendered from the camera perspective to produce a synthetic RGB image. Since the viewpoint is known during rendering, the corresponding camera location is also recorded.

This process is repeated until the required number of training samples has been generated.

### 4.4.2. Automatic Sky Mask Generation

The sky mask is generated directly from the rendered scene. Since the terrain model is completely known, the system can identify whether each pixel belongs to the sky or the terrain.

The generated mask is stored as a binary image. Sky pixels are assigned a value of 0, while terrain pixels are assigned a value of 255. Every RGB image has one corresponding binary mask, producing perfectly aligned training pairs without manual annotation.

The generated masks are later used as the ground truth labels during segmentation model training.

### 4.4.3. Ground Truth Storage

Each generated sample is accompanied by its corresponding viewpoint information. The dataset stores the RGB image, binary mask, and camera information using the same sample identifier.

The generated images and masks are stored separately, while the viewpoint information is recorded in a ground truth file. This organization allows the images, masks, and viewpoint information to be loaded independently during model training and system evaluation.

## 4.5. Skyline Segmentation

The first step of localization is separating the sky from the surrounding terrain. The resulting binary mask is used to extract the skyline profile for matching. Since mountain images often contain snow, shadows, clouds, and changing illumination, simple image processing techniques do not produce reliable results. A deep learning model is used to perform the segmentation.

To improve generalization beyond purely synthetic scenes, the segmentation stage is also informed by examples from the GeoPose3K benchmark [9]. GeoPose3K provides real mountain photographs with known camera poses, making it useful for exposing the model to realistic skyline boundaries, lighting variation, and terrain appearance that are difficult to capture through rendering alone.

The segmentation process consists of three stages. First, the input image is processed by a U-Net model with a MobileNetV3 encoder to predict the sky region. Next, the predicted mask is converted into a binary image. Finally, connected component analysis and edge refinement are applied to improve the skyline boundary.

U-Net was selected instead of DeepLabV3+ because skyline boundary accuracy is critical for matching. The task is binary segmentation, so a lighter decoder is sufficient. Skip connections preserve edge detail from shallow layers and help keep ridge boundaries sharp. This improves alignment with reference skylines while keeping computation lower.

MobileNetV3 was selected for smartphone-oriented deployment. It uses low memory and supports fast inference. At the same time, it provides good segmentation accuracy for this task.

Figure 4-5 illustrates the segmentation process.

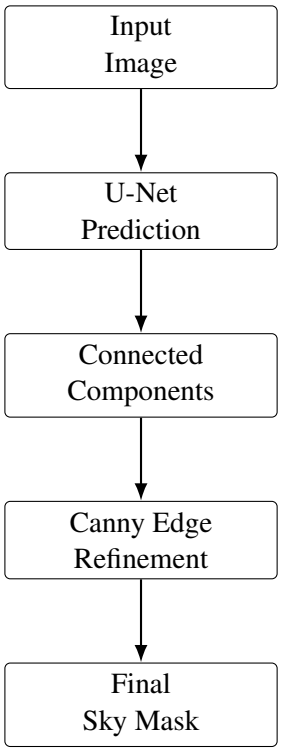



Figure 4-5. Skyline segmentation process.

### 4.5.1. Sky Segmentation

The input image is resized before being passed to the segmentation model. The model predicts the probability that each pixel belongs to the sky. The predicted probability map is converted into a binary mask using a fixed threshold.

This produces an initial estimate of the sky region. Although the overall sky boundary is usually detected correctly, small segmentation errors may still appear around mountain ridges.

### 4.5.2. Mask Refinement

The predicted mask is refined before skyline extraction. The first refinement step removes disconnected sky regions using connected component analysis. Only sky regions connected to the top boundary of the image are retained.

The skyline boundary is then refined using Canny edge detection. For every image column, the detected skyline is adjusted to the nearest edge within a local search window. This improves the alignment between the predicted skyline and the visible mountain ridge.

The final output is a binary image where sky pixels have a value of 0 and terrain pixels have a value of 255. This mask is passed to the skyline extraction stage.

## 4.6. Skyline Extraction

The segmented sky mask is converted into a one-dimensional skyline profile before localization. Instead of comparing binary images directly, the system represents the skyline as a sequence of elevation angles. This representation is independent of image resolution and can be compared directly with the reference skyline database.

The skyline extraction process consists of three steps. First, the skyline boundary is detected from the binary mask. Next, the detected boundary is converted into elevation angles using the camera model. Finally, the resulting profile is validated before matching.

Figure 4-6 shows the skyline extraction process.

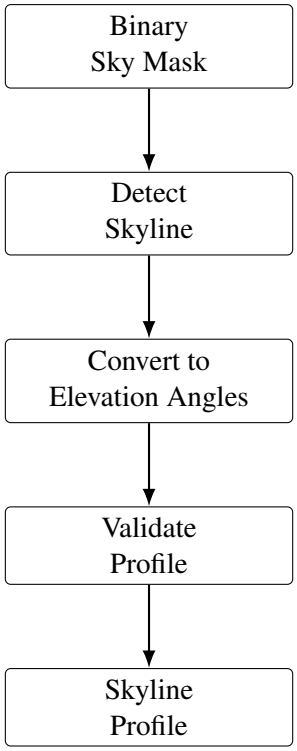



Figure 4-6. Skyline extraction process.

### 4.6.1. Skyline Detection

The skyline is extracted independently for every image column. Starting from the top of the binary mask, the first terrain pixel is identified and recorded as the skyline position. If no terrain pixel is found in a column, the bottom of the image is used.

A median filter with a window size of 5 is applied to remove isolated noise while preserving the skyline shape.

### 4.6.2. Elevation Angle Computation

The detected skyline is represented as image coordinates. These coordinates are converted into viewing rays using the camera field of view and image dimensions.

The elevation angle of each viewing ray is computed as

$$\theta = \sin^{-1}(r_y), \tag{4-2}$$

where $r_y$ is the vertical component of the normalized viewing ray.

Elevation angles are used instead of pixel coordinates because they are independent of image resolution. They also provide a common geometric representation for direct comparison with reference skylines.

The corresponding azimuth angle is computed as

$$\phi = \tan^{-1}\left(\frac{r_x}{-r_z}\right), \tag{4-3}$$

where $r_x$ and $r_z$ are the horizontal components of the viewing ray.

The skyline profile is interpolated onto a uniform azimuth grid before matching. Uniform sampling ensures that every query profile can be compared directly with the reference skyline database.

### 4.6.3. Profile Validation

Some images contain very little visible terrain. These images produce skyline profiles with almost no variation and are unsuitable for localization.

Before matching, every extracted profile is checked for profile quality. The check uses simple geometric characteristics, such as skyline variability and peak prominence. Nearly flat skylines are discarded because they contain little discriminative information.

## 4.7. Skyline Matching

The extracted skyline profile is compared with the reference skyline database to estimate the camera location. Since the database contains a large number of skyline profiles, the matching process is divided into two stages. The first stage removes unlikely candidates using fast correlation, while the second stage performs detailed alignment on the remaining candidates. The direction in which the input image was taken is usually unknown. Therefore, the system searches for the orientation that produces the best match between the query skyline and each reference skyline.

Figure 4-7 illustrates the skyline matching process.

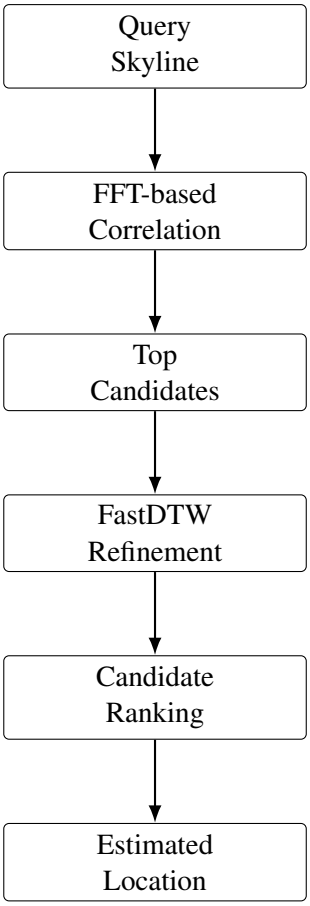



Figure 4-7. Skyline matching process.

### 4.7.1. Feature Extraction

The skyline profile alone does not fully describe the terrain shape. To capture both the overall skyline and local changes, three feature sequences are computed from every profile. These consist of the normalized skyline profile together with its first and second derivatives.

Each feature is normalized before matching. This reduces the effect of differences in elevation magnitude while preserving the overall skyline shape.

### 4.7.2. Candidate Selection

Because the viewing direction of the query image is generally unknown, the query skyline is compared against all possible azimuth orientations of each reference skyline. If smartphone compass information is available, the search can be restricted to headings close to the measured orientation.

The first stage compares the query skyline with every reference skyline using cross-correlation. The correlation scores obtained from the three feature sequences are combined into a single similarity score.

Because a query photo covers only a limited field of view while each reference skyline covers 360°, the query profile is evaluated at multiple azimuth offsets over the full reference profile. This sliding-window correlation step finds the best matching azimuth window before detailed refinement.

The best correlation also provides the rotational alignment between the query skyline and the reference skyline. If compass information is available, only rotations close to the measured heading are considered. This reduces the search space and removes unlikely candidates.

To reduce computation, an initial search is performed using a coarse spatial grid containing every Nth viewpoint. Neighbouring viewpoints are then evaluated at full resolution.

Height filtering is used as an optional preliminary stage during coarse candidate filtering. Smartphone altitude from barometer or GNSS provides an approximate elevation. Viewpoints whose elevations differ substantially are discarded before skyline matching. This reduces the number of candidates, and Chapter 6 evaluates its effect on localization performance and computation time.

### 4.7.3. Profile Alignment

The selected candidates are refined using Dynamic Time Warping (DTW). Unlike direct point-by-point comparison, DTW allows small local differences between skyline profiles while preserving the overall terrain shape. The implementation uses a Sakoe-Chiba banding constraint to limit the warping path, which significantly reduces computation time while remaining effective for comparing closely aligned terrain features.

The alignment cost produced by DTW measures the similarity between the query skyline and each candidate skyline. Lower alignment costs indicate better matches.

### 4.7.4. Candidate Ranking

Each candidate contains a correlation score and a DTW alignment cost. These two measures are combined to rank the candidate locations.

The candidate with the highest overall similarity is selected as the final localization result. The coordinates associated with the selected reference skyline are returned as the estimated camera location.

# 5. IMPLEMENTATION DETAILS

This chapter describes the implementation of the proposed visual geolocation system. The system is implemented as a sequence of software modules. Each module performs one specific task and passes its output to the next stage.

The complete implementation contains five main components. These are the reference skyline database generation, synthetic dataset generation, sky segmentation, skyline extraction, and skyline matching modules. The final evaluation module measures the localization performance using generated query images and known ground truth locations.

## 5.1. Reference Skyline Database Implementation

The reference skyline database provides the collection of possible terrain skyline profiles used during localization. Each database entry represents a possible camera viewpoint inside the study area.

The database generation module takes the DEM file as input and samples viewpoints on a grid. Here, two distances must be separated clearly. The DEM raster has an approximate spatial resolution of 30 m, but the evaluation database used in this report is sampled at a 500 m viewpoint interval. The 30 m setting was used in initial local tests and sensitivity checks. The reported Chapter 6 results use the 500 m grid.

The 500 m viewpoint spacing was selected as a practical balance. It reduces computational cost, RAM usage, and database size while keeping matching speed high. In the Himalayas, skyline shape usually changes gradually over this distance, so the grid still preserves useful spatial detail.

For every viewpoint, the surrounding terrain is analysed up to a maximum distance of 30 km. The visible terrain boundary is converted into a one-dimensional skyline profile.

The 30 km horizon radius was selected for this region because Himalayan valleys are strongly affected by terrain shielding. Beyond this range, Earth's curvature and atmospheric haze reduce visibility, and distant ridges add little stable detail. Viewshed analysis also showed that most useful skyline structure is already captured within 30 km.

The skyline is stored as elevation angles sampled at 1° azimuth intervals. Therefore, each viewpoint contains a profile with 360 elevation values representing the complete horizontal view around the camera.

For each viewpoint, the terrain horizon is generated by finding the maximum elevation angle along each viewing direction. The elevation-angle computation follows Eq. (4-1).

In this chapter, we use the shorthand $h = h_t - h_o$, so the same relation is written as $\theta = \tan^{-1}(h/d)$.

Not all generated viewpoints provide useful localization information. Flat regions and open areas can produce similar skyline shapes at many locations. To reduce these ambiguous cases, a profile-quality screening step is applied before profiles are stored.

This screening step removes profiles with very weak skyline information and keeps profiles with clearer terrain structure for matching.

The generated database is stored using the Parquet file format. Each record contains the viewpoint position and its corresponding skyline profile. The database format allows the matching module to access required rows without loading the complete dataset into memory.

### 5.2. Synthetic Dataset Generation

A synthetic dataset is generated to train and evaluate the skyline segmentation module. The dataset generation process creates mountain images together with their corresponding sky masks and camera information.

The terrain model is generated from the DEM. Satellite texture data is projected onto the terrain surface to provide realistic ground appearance. The rendering process produces images from different camera viewpoints selected from the terrain.

To increase image variation, different environmental conditions are applied during generation. The implemented presets include sunny, cloudy, sunset, storm, rainy, and night conditions. Each weather preset is generated by varying the lighting and sky colors, while procedural effects such as a randomly placed glowing sun, randomly placed star points, and random rain streaks are added where appropriate. Additional image effects such as sensor noise, vignette, lens flare, and chromatic effects are applied to simulate variations that may appear in real images.

The camera locations are randomly shifted from the original viewpoint grid positions. The displacement ranges from approximately ±1 m to ±15 m. This prevents the matching system from relying on exact database grid positions and allows evaluation of nearby locations.

For every rendered image, a corresponding binary sky mask is generated. The mask separates the sky region from the terrain region. Since the rendering environment is fully known, the generated masks provide accurate labels for model training.

The generated samples are stored in separate image and mask folders. The ground truth camera coordinates are stored separately and are used during evaluation.

Only 300 synthetic images were needed in this phase because the encoder was pretrained and did not start from scratch. Data augmentation increased visual diversity during training, and the synthetic set mainly supported domain adaptation to local Himalayan terrain appearance.

### 5.3. Sky Segmentation Implementation

The sky segmentation module extracts the sky region from a query image. The output of this module is a binary mask containing the sky and terrain boundary.

The segmentation model is implemented using a U-Net architecture with a MobileNetV3 encoder. The model receives an RGB image as input and produces a probability map representing the likelihood of each pixel belonging to the sky region.

U-Net was selected instead of DeepLabV3+ because this is a binary segmentation task and skyline boundary accuracy is critical. Its skip connections preserve edge detail from early layers, which helps keep ridge boundaries sharp. It also has lower computational cost for this use case.

MobileNetV3 was selected for smartphone deployment. It has low memory use and fast inference while maintaining good accuracy. This makes it suitable for offline field use on limited hardware.

Before inference, each image is resized to $256 \times 256$ pixels. The pixel values are normalized using the standard image normalization parameters used during model training.

The model output is converted into a binary mask using a threshold value of 0.5. The resulting mask is resized back to the original image resolution.

The initial segmentation result may contain small errors near the mountain boundary. These errors can occur because snow, clouds, and bright terrain regions may have similar appearance to the sky. To improve the boundary accuracy, an edge refinement process is applied.

First, connected component analysis is used to keep only sky regions connected to the top boundary of the image. This removes isolated incorrect regions.

Next, Canny edge detection is applied to the input image. The detected edges are used to adjust the predicted boundary towards strong terrain edges. A local search window is used around the predicted skyline position to find the closest edge.

The final refined mask is stored for skyline extraction.

### 5.4. Skyline Extraction Implementation

The skyline extraction module converts the binary sky mask into a one-dimensional elevation profile. This profile is later compared with the reference database.

The extraction process begins by detecting the sky-terrain boundary for every image column. The first terrain pixel from the top of each column is selected as the skyline position.

A median filter with a window size of five is applied to remove small discontinuities caused by segmentation noise.

The detected pixel coordinates are converted into viewing rays using the camera field of view and image dimensions. The horizontal field of view is calculated from the vertical field of view and image aspect ratio.

The normalized camera ray is represented as

$$\mathbf{r} = \begin{bmatrix} r_x \\ r_y \\ r_z \end{bmatrix}. \tag{5-1}$$

The elevation and azimuth angles are computed using Eqs. (4-2) and (4-3), respectively, as defined in Section 4.6.2.

The extracted profile is then interpolated onto a uniform azimuth grid with the same sampling interval as the database profiles.

Before matching, the profile is checked for sufficient terrain variation. Profiles with very weak skyline structure are rejected because they do not contain enough information for reliable localization.

### 5.5. Skyline Matching Implementation

The skyline matching module estimates the camera location by comparing the extracted query profile with the reference database.

The matching process uses a two-stage search strategy. The first stage performs a fast similarity search to identify promising candidates. The second stage performs detailed alignment on the selected candidates.

**Algorithm 2** Skyline Extraction from Binary Sky Mask

**Require:** Binary sky mask $M$, camera field of view, image dimensions
**Ensure:** Skyline profile (elevation angle vs. azimuth)
**for all** image columns $c$ **do**
    Scan from top of column to find first terrain pixel
    **if** no terrain pixel found in column **then**
        Set skyline position to bottom of image
    **end if**
**end for**
Apply median filter (window size 5) to remove discontinuities
**for all** detected skyline pixels **do**
    Convert pixel coordinate to normalized viewing ray $\mathbf{r} = (r_x, r_y, r_z)$
    Compute elevation angle $\theta = \sin^{-1}(r_y)$
    Compute azimuth angle $\phi = \tan^{-1}(r_x/-r_z)$
**end for**
Interpolate profile onto uniform azimuth grid
**if** profile variation is below minimum threshold **then**
    Reject profile as unsuitable for matching
**end if**
**return** the validated skyline profile

The query skyline comes from a single camera view with limited field of view, while each reference profile covers 360°. In the coarse stage, the query is slid across the full 360°reference profile and cross-correlation is computed at different azimuth offsets. The best offset gives the most likely viewing direction window before fine refinement.

During the first stage, the database is processed in chunks of 4000 rows. This avoids loading the complete database into memory and keeps the memory usage low.

The skyline profiles are compared using Fast Fourier Transform (FFT)-accelerated cross-correlation. Three feature representations are extracted from each profile:

FFT matching was selected instead of normalized cross-correlation in the spatial domain because it provides fast coarse filtering when camera heading is unknown. The query must be tested over many azimuth offsets, and all profiles are already z-score normalized, so this approach is computationally efficient for large candidate sets.

1. Normalized skyline elevation profile.
2. First derivative of the profile.
3. Second derivative of the profile.

The combined correlation score is calculated using equal weights:

$$S = \frac{1}{3}\left(S_1 + S_2 + S_3\right). \tag{5-2}$$

The highest scoring candidates are selected after the coarse search. To reduce computation, only every fifth viewpoint is initially checked.

The neighbouring viewpoints around the best coarse candidates are then selected for fine refinement. Full resolution skyline profiles are loaded only for these candidates.

The final refinement stage uses DTW to compare the query profile and candidate profiles. DTW allows small local shifts between skyline features while preserving the overall shape. A Sakoe-Chiba banding constraint is applied to the warping path, which restricts alignment to a narrow diagonal band around the optimal path. This constraint significantly improves runtime performance while remaining effective for matching similar skyline profiles.

DTW was selected instead of Euclidean distance because skyline comparison needs local alignment. Small shifts from segmentation noise and DEM discretization can misalign peaks by a few samples. DTW is more robust to these local shifts while preserving global shape consistency.

Each candidate receives a final score based on its correlation value and DTW alignment cost. The candidate with the best combined score is selected as the estimated camera location.

### 5.6. Evaluation Implementation

The evaluation module measures localization accuracy using synthetic query images with known ground truth coordinates.

For each query image, the predicted viewpoint coordinates are compared with the true camera coordinates. The distance error is calculated using geodesic distance.

The system reports several evaluation metrics:

- Top-1 accuracy within 500 m.
- Top-1 accuracy within 100 m.
- Top-5 accuracy within 500 m.
- Median localization error.
- Mean localization error.

**Algorithm 3** Two-Stage Skyline Matching

**Require:** Query skyline profile $Q$, reference skyline database $D$, feature weights $w_1, w_2, w_3$

**Ensure:** Estimated camera location

Compute normalized profile, first derivative, and second derivative of $Q$

▷ Stage 1: Coarse candidate selection

**for all** reference viewpoints $v \in D$ (every 5th viewpoint) **do**

    Slide $Q$ across the full 360° reference profile of $v$

    Compute FFT-accelerated cross-correlation at each azimuth offset

    Combine correlation scores: $S = w_1 S_1 + w_2 S_2 + w_3 S_3$

    ▷ $w_1, w_2, w_3$ set to $\frac{1}{3}$ in current implementation

    Record best azimuth offset and combined score for $v$

**end for**

Select top-$K$ candidates by combined correlation score

▷ Stage 2: Fine alignment

**for all** selected candidates and their neighbouring viewpoints **do**

    Load full-resolution reference skyline profile

    Compute DTW alignment cost between $Q$ and candidate profile

**end for**

Combine correlation score and DTW alignment cost into final rank

Select candidate with best combined score

**return** coordinates of the selected candidate

# 6. RESULTS AND ANALYSIS

## 6.1. Evaluation Setup

We evaluated the full pipeline on synthetic query images from the Khumbu region. We used synthetic queries here because they give us exact ground truth camera locations, so location error can be measured directly without manual labelling.

For this preliminary evaluation stage, we used a reference database sampled on a regular 500 m grid, which produced 4,920 viewpoints in the selected study area. Each viewpoint stores a precomputed skyline profile from the same DEM used in the database generation step.

To avoid confusion: 30 m refers to DEM raster resolution, while 500 m is the viewpoint sampling interval used in this initial Chapter 6 setup. This grid setting is not treated as final and will be refined in later stages.

Table 6-1. Evaluation setup used in this chapter.

| Parameter | Value |
|---|---|
| Study area | Khumbu region, Nepal |
| Reference viewpoints (preliminary setup) | 4,920 |
| Grid spacing (preliminary setup) | 500 m |
| Query images | 300 |
| Query type | Synthetic, grid-aligned |
| Ground truth | Known camera location |

All metrics reported in this chapter are measured under this preliminary setup: 300 synthetic queries, evaluated against the 500 m reference grid used for this stage.

For this evaluation, skyline profiles are extracted using the ground-truth synthetic sky masks (Section 4.4.2) rather than masks predicted by the trained segmentation model. This isolates the accuracy of the skyline extraction and matching stages from segmentation error. End-to-end evaluation using predicted masks is planned as future work.

Figure 6-1 shows an example view from the reference skyline database.

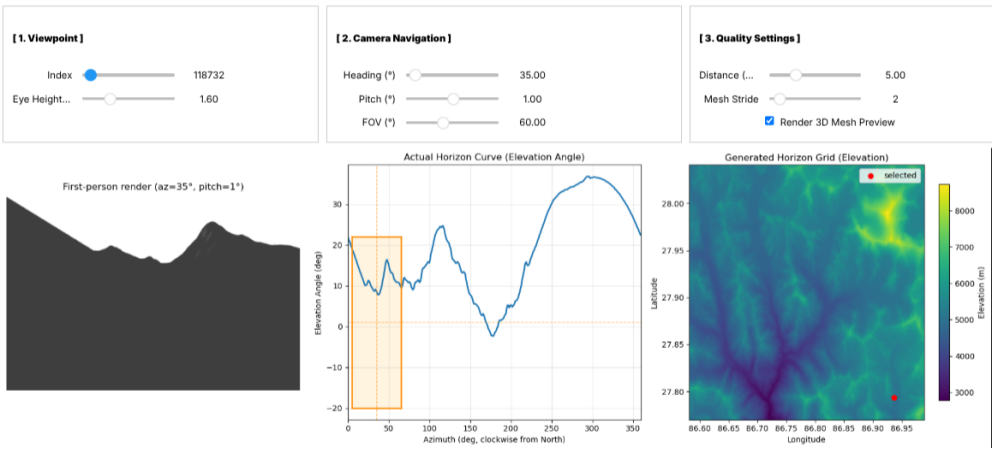
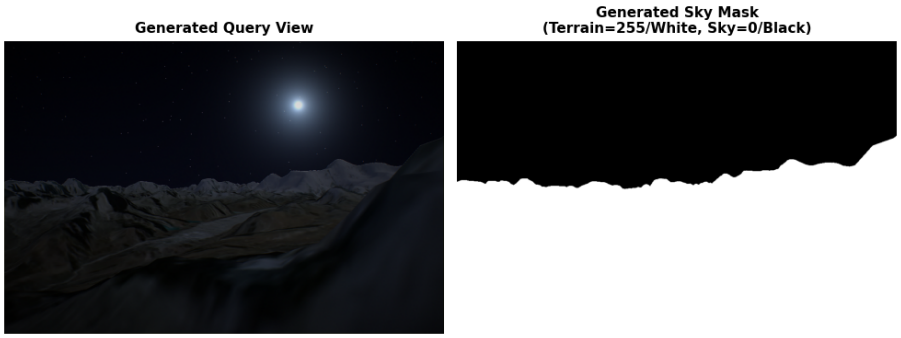



Figure 6-1. Reference skyline database preview used during matching.

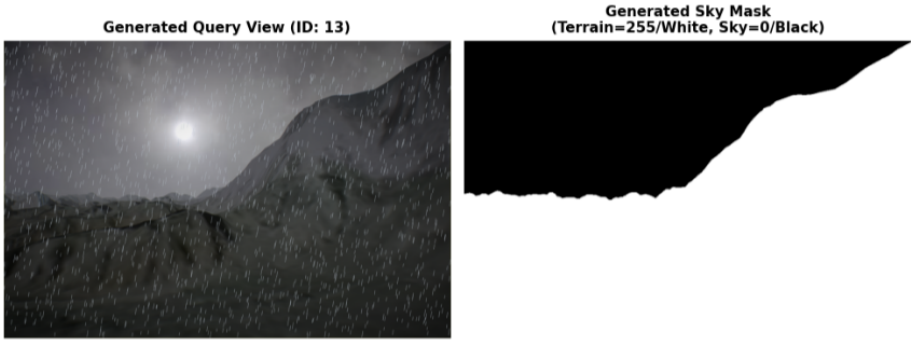
## 6.2. Synthetic Query Images

The query set was generated by rendering synthetic mountain scenes under different lighting conditions and simple weather effects. This gives controlled variation while keeping exact camera metadata.

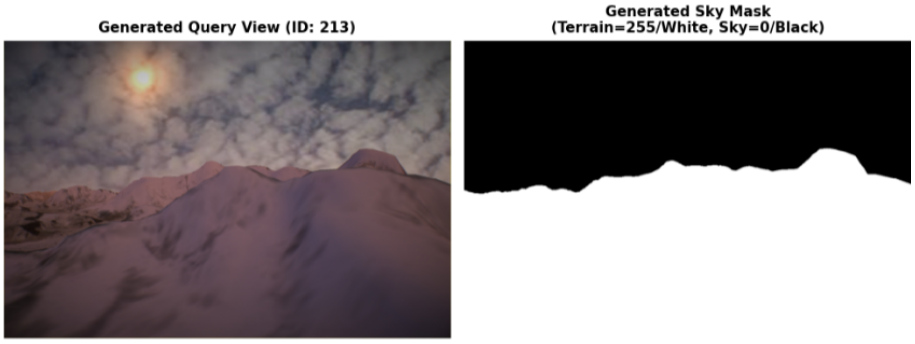
Figure 6-2 shows sample synthetic query images used in testing.




(a) Synthetic sample A




(b) Synthetic sample B




(c) Synthetic sample C

Figure 6-2. Examples of synthetic query images used for evaluation.

### 6.3. Segmentation Training Result

We trained the skyline segmentation model for 15 epochs using GeoPose3K's predefined training split combined with our 300 synthetic images. We used transfer learning with a pretrained MobileNetV3 encoder. Figure 6-3 shows the training and validation curves from that run.

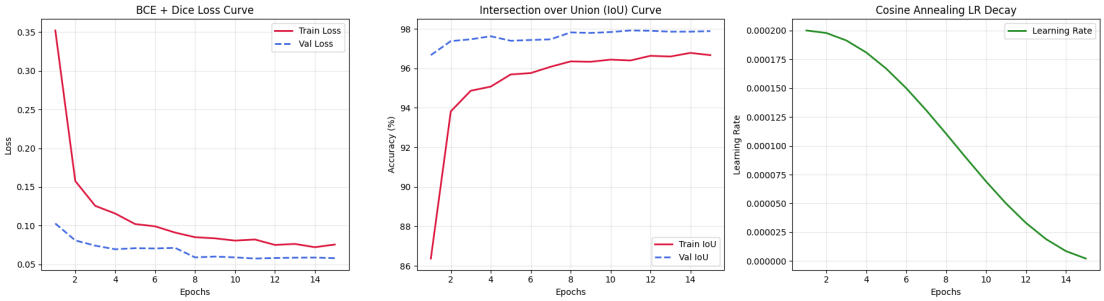



Figure 6-3. Training and validation curves of the segmentation model.

Both training and validation loss decrease over epochs, and validation Intersection over Union (IoU) stays high near the end of training. This means the model is learning the sky-terrain boundary under our mixed training setup (synthetic + GeoPose). During training, we also applied data augmentation to increase sample diversity. It still does not by itself prove the same quality on all real field photos, where haze, exposure, and foreground objects can differ from training data.

## 6.4. Localization Accuracy

Table 6-2 reports localization accuracy for the same synthetic, grid-aligned query set described in Section 6-1.

Table 6-2. Localization accuracy of the proposed system.

| Metric | Result |
|---|---|
| Accuracy @ 10 m | 84.00% |
| Accuracy @ 100 m | 84.00% |
| Accuracy @ 500 m | 90.33% |
| Accuracy @ 1000 m | 94.67% |
| Median error | 0.0 m |
| Failed matches (>1000 m) | 5.33% |

*Note:* Accuracy@10m and @100m are identical because predictions in this grid-aligned setup either land exactly on the correct viewpoint or miss by more than 100 m (see Section 6.4.1). Median error of 0.0 m reflects this same grid-alignment effect, not zero-error performance in general.

Figure 6-4 shows the same values as a plot.

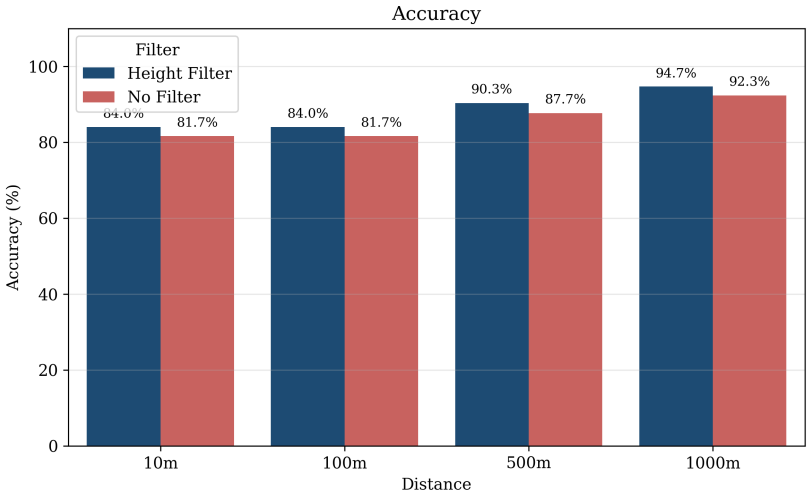



Figure 6-4. Localization accuracy at different distance thresholds.

Figure 6-4 also includes the result for optional height-filtered candidate pruning. In this experiment, reference viewpoints were restricted to a vertical window of ±100 m around the query elevation before skyline matching. Under the same grid-aligned synthetic setup, Top-1 accuracy at the 1000 m threshold increased from 92.3% to 94.7%. This shows height filtering as a preliminary filtering stage that was experimentally evaluated, rather than the core localization method.

Out of 300 synthetic grid-aligned queries, 284 are localized within 1000 m, 271 within 500 m, and 252 within 100 m.

The 10 m and 100 m values are both 84.00% because of the test condition. In this dataset, many predictions either land exactly on the correct grid viewpoint or miss by more than 100 m, so both thresholds count the same 252 samples.

The median error is 0.0 m under this setup because at least half of the queries map exactly to the correct grid point. This is a side effect of grid-aligned testing, not a claim of zero-error behavior in free-position real-world use.

### 6.4.1. Result Interpretation

Figure 6-4 and the qualitative examples in Figure 6-5 are both measured on grid-aligned synthetic queries sampled from the same grid family as the reference database. These results therefore represent a favorable matching condition. Off-grid accuracy is not measured in this chapter.

If a user stands between grid points (for example, near the midpoint between neighboring samples), localization can degrade because no reference point is exactly at that location. However, distant mountain peaks are usually several kilometers away, so moderate

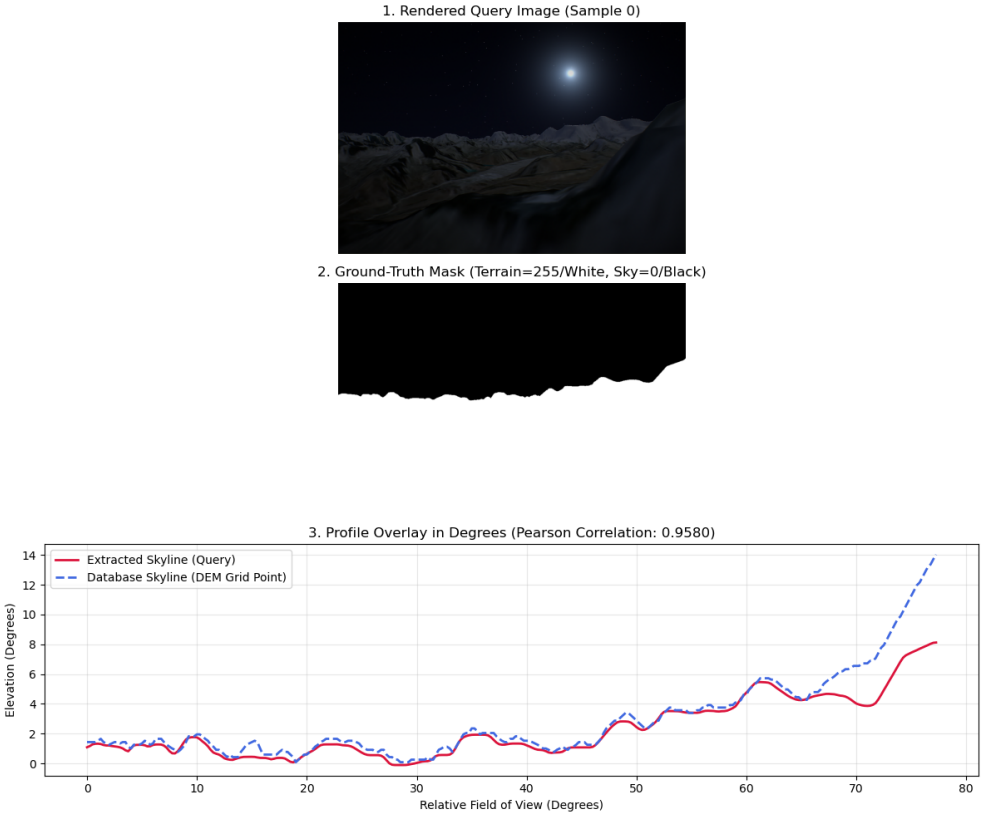
horizontal shifts produce limited angular skyline change. The DTW refinement step helps handle these small local distortions by allowing non-rigid alignment rather than strict point-to-point matching.

### 6.4.2. Evaluation Limitations

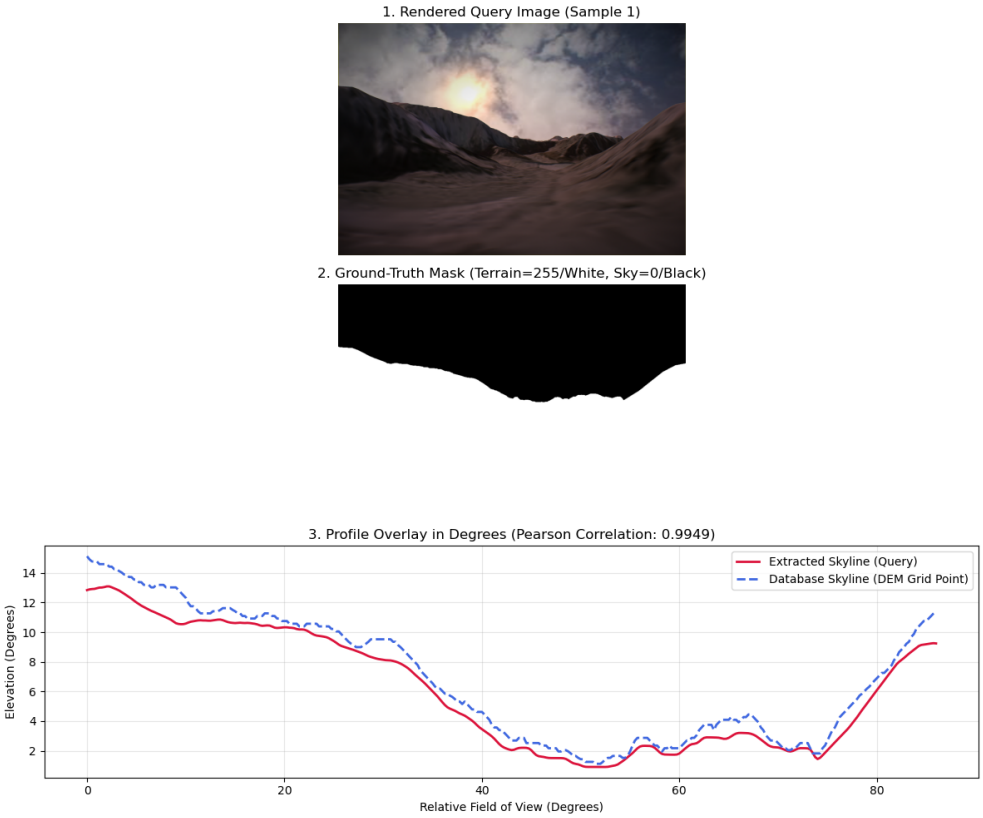
This evaluation has four key limitations. First, all queries are grid-aligned synthetic views, so off-grid behavior is not measured. Second, camera tilt is not varied; all queries use horizontal orientation, while real users often tilt the camera up or down. Third, the images are synthetic with relatively clear sky boundaries, whereas real photos may include haze, cloud, and foreground clutter that make skyline extraction harder. Fourth, localization accuracy is evaluated using ground-truth sky masks rather than segmentation model predictions, so these results do not include segmentation error. Combining the trained segmentation model with the matching pipeline for full end-to-end evaluation is the next planned step. A deeper error analysis across terrain and image conditions remains future work.

## 6.5. Qualitative Examples

Figure 6-5 shows two representative localization outputs.




(a) Sample 0




(b) Sample 1

Figure 6-5. Representative localization visualizations from the grid-aligned test set.

In successful cases, the predicted skyline shape matches the reference shape at the main peaks and valleys. In failed cases, the skyline still looks visually similar, but a nearby wrong candidate receives a higher score.

The failure rate in this test is 5.33% (16 out of 300, from the "Failed matches (>1000 m)" row in Table 6-2). In inspected failed examples, two patterns are commonly observed:

1. Low skyline contrast in difficult weather or lighting, which makes boundary extraction less stable.
2. Repeated ridge geometry, where different viewpoints produce similar skyline profiles.

This explains why some failures are not random noise but structured confusion between look-alike terrain profiles.

### 6.6. Summary

The system performs well on this synthetic grid-aligned benchmark using ground-truth sky masks: 94.67% of queries are within 1 km and 90.33% are within 500 m; end-to-end accuracy with predicted segmentation masks has not yet been measured. Because the database spacing is 500 m, a 1 km error corresponds to approximately two grid cells. This helps interpret the 1000 m metric in terms of the reference sampling geometry.

These numbers should not be read as final field performance. They are measured on synthetic data with known camera states and grid-aligned viewpoints. Real-world testing will need off-grid captures, and stronger environmental variation before we can claim deployment-level accuracy.

# 7. REMAINING TASKS

The project has produced a working prototype for skyline-based visual geolocation in the Khumbu region. The next phase focuses on validation and reliability improvements for practical deployment.

## 7.1. Google Street View Validation

The current evaluation uses synthetic images generated from the terrain model. While this provides accurate ground-truth locations, synthetic images cannot fully represent real photographs.

The next stage of this work is to evaluate the system using Google Street View (GSV) images. These images contain challenges such as haze, shadows, foreground objects, and changing lighting conditions that are difficult to reproduce synthetically. The Everest Base Camp trekking route already has Google Street View coverage, so real-world validation can be performed without collecting new field data.

Comparing these results with the synthetic benchmark will clarify how performance changes under real-world conditions.

## 7.2. Automatic Failure Detection

The system currently always returns a location estimate, even when the extracted skyline contains too little information for reliable matching.

Future work will investigate methods for detecting poor-quality skyline profiles before localization. Images with heavy occlusion, weak skyline structure, or large foreground objects may be rejected automatically instead of producing unreliable results.

This would improve the reliability of the system when processing difficult images.

## 7.3. Landmark Association

The system currently returns the estimated camera location and viewing direction. Future work will integrate geographic landmark databases to provide additional context about the surrounding terrain.

After localization, visible mountain peaks and other landmarks could be identified automatically from the estimated viewpoint. This would make the localization results easier to interpret and verify.

### 7.4. Off-Grid Robustness

The current quantitative evaluation is based on grid-aligned synthetic queries. Future evaluation should therefore include off-grid queries where the true camera position lies between reference viewpoints, which better reflects real user positions in the field.

In this off-grid setting, height filtering can be used to reduce ambiguity between similar skyline shapes at different altitudes.

Because current filtering results are preliminary and measured on grid-aligned data, future work should verify whether the same trend holds when off-grid queries and realistic elevation uncertainty are introduced.

### 7.5. Summary

The remaining work emphasizes real-world validation and stronger reliability in difficult conditions. Off-grid robustness is a key next step, with height filtering treated as a supporting technique within that broader evaluation. Together, these tasks will strengthen practical usability while preserving the core localization framework.

# A. PROJECT TIMELINE

## A.1. Gantt Chart

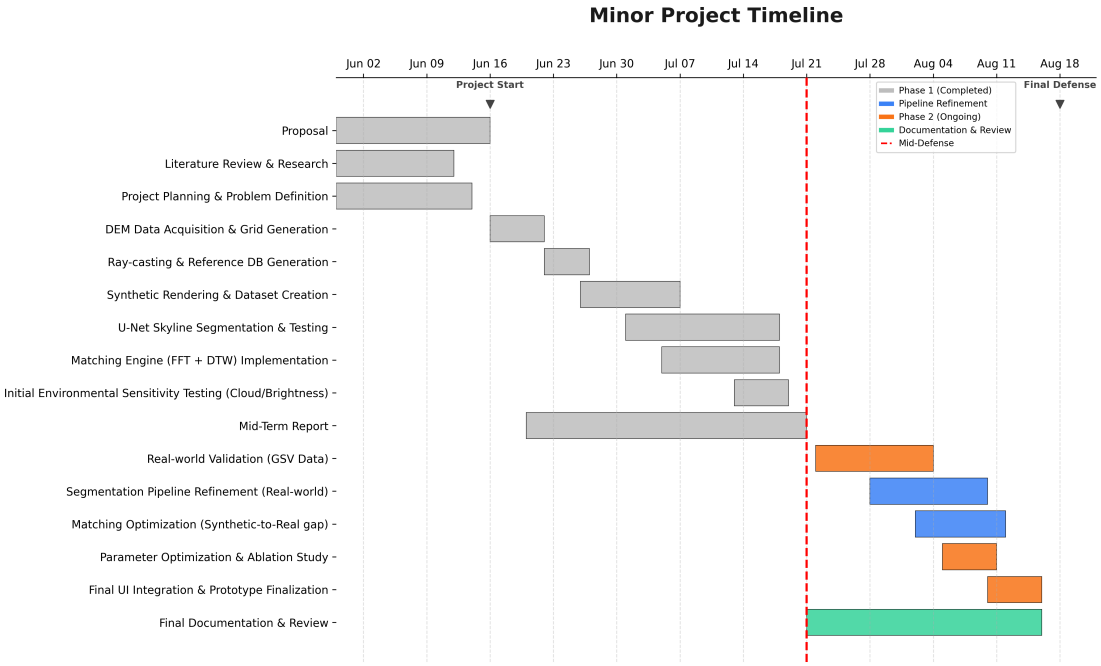



Figure A-1. Project Gantt Chart

## REFERENCES


[1] P. C. Naval, M. Mukunoki, M. Minoh, and K. Ikeda, "Estimating camera position and orientation from geographical map and mountain image," in *38th Pattern Sensing Group Research Meeting, Society of Instrument and Control Engineers*, 1997, pp. 9–16.

[2] P. C. Naval, "Camera pose estimation by alignment from a single mountain image," in *International Symposium on Intelligent Robotic Systems*, 1998, pp. 157–163.

[3] G. Baatz, O. Saurer, K. Köser, and M. Pollefeys, "Large scale visual geo-localization of images in mountainous terrain," in *European Conference on Computer Vision (ECCV)*, 2012, pp. 517–530.

[4] O. Saurer, G. Baatz, K. Köser, L. Ladický, and M. Pollefeys, "Image based geo-localization in the Alps," *International Journal of Computer Vision*, vol. 116, no. 3, pp. 213–225, 2016.

[5] H. Li, J. Zhao, B. Yan, L. Yue, and L. Wang, "Global DEMs vary from one to another: An evaluation of newly released Copernicus, NASA and AW3D30 DEM on selected terrains of China using ICESat-2 altimetry data," *International Journal of Digital Earth*, vol. 15, no. 1, pp. 1149–1168, 2022.

[6] Y. Chen, G. Qian, K. Gunda, H. Gupta, and K. Shafique, "Camera geolocation from mountain images," in *18th International Conference on Information Fusion (Fusion)*. IEEE, 2015, pp. 1587–1596.

[7] Z. Pan, J. Tang, T. Tjahjadi, X. Xiao, and Z. Wu, "Camera geolocation using digital elevation models in hilly area," *Applied Sciences*, vol. 10, no. 19, p. 6661, 2020.

[8] Z. Pan, J. Tang, T. Tjahjadi, and F. Guo, "Fast geo-location method based on panoramic skyline in hilly area," *ISPRS International Journal of Geo-Information*, vol. 10, no. 8, p. 537, 2021.

[9] J. Brejcha and M. Čadík, "GeoPose3K: Mountain landscape dataset for camera pose estimation in outdoor environments," *Image and Vision Computing*, vol. 66, pp. 1–14, 2017.

[10] S. W. Yang, I. C. Kim, and J. S. Kim, "Robust skyline extraction algorithm for mountainous images," in *Proceedings of the Second International Conference on Computer Vision Theory and Applications (VISAPP)*, 2007, pp. 253–257.

[11] T. Ahmad, E. Emami, M. Čadík, and G. Bebis, "Resource efficient mountainous skyline extraction using shallow learning," in *International Joint Conference on*

*Neural Networks (IJCNN)*. IEEE, 2021, pp. 1–9.

[12] J. Tomešek, M. Čadík, and J. Brejcha, "CrossLocate: Cross-modal large-scale visual geo-localization in natural environments using rendered modalities," in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2022, pp. 3174–3183.

[13] Z. Liu, F. Guo, H. Liu, X. Xiao, and J. Tang, "CMLocate: A cross-modal automatic visual geo-localization framework for a natural environment without GNSS information," *IET Image Processing*, vol. 17, no. 12, pp. 3524–3540, 2023.